

Maven Project To Connect To Web Servers Using Selenium Dependency

Brief Introduction

A Maven project using Selenium dependency is used to automate web browser operations and connect to web servers or websites for testing purposes.

In this project, Apache Maven manages all required Selenium libraries automatically through the pom.xml file. Selenium allows the program to open a web browser, access websites, perform actions such as clicking buttons or entering text, and retrieve webpage information.

This type of project is commonly used in DevOps and software testing for:

- Automated web application testing
- Browser automation
- Continuous Integration and Continuous Deployment (CI/CD)
- Website functionality verification

Working Process

1. Maven downloads Selenium dependencies automatically.
2. Selenium WebDriver launches the browser.
3. The browser connects to the required web server or website.
4. Automated actions and testing are performed.
5. Results are displayed and the browser closes.

Advantages

- Reduces manual testing effort
- Supports automated testing
- Easy dependency management
- Integrates with CI/CD tools like Jenkins
- Improves software quality and testing speed

This project demonstrates how Maven and Selenium work together to automate web-based operations efficiently.

Steps to execute the Project:

Step1:

Before beginning the project building, install ChromeDriver.

- Check the Google Chrome version: `google-chrome --version` Example output: Google Chrome 133.0.6943.141
- Extract the Chrome version and store it in a variable: `CHROME_VERSION=$(google-chrome --version | awk '{print $3}' | cut -d'.' -f1-3)`
- Install jq (used to process JSON data): `sudo apt install jq`
- If curl is not installed, install it using: `sudo apt install curl`
- Get the latest compatible ChromeDriver version: `LATEST_DRIVER=$(curl -s "https://googlechromelabs.github.io/chrome-for-testing/latestpatch-versions-per-build.json" | jq -r ".builds[\"$CHROME_VERSION\"].version")`
- Download ChromeDriver: `wget https://storage.googleapis.com/chrome-for-testing-public/$LATEST_DRIVER/linux64/chromedriver-linux64.zip`

- Extract the downloaded file: `unzip chromedriver-linux64.zip`
 - Move ChromeDriver to the system path: `sudo mv chromedriver-linux64/chromedriver /usr/local/bin/chromedriver`
 - Give execution permission: `sudo chmod +x /usr/local/bin/chromedriver`
 - Verify the installation: `chromedriver --version`
 - The output version should match the installed Chrome version
- Step 2:**

```

Activities Terminal Mar 4 21:38
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp01

vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp01$ google-chrome --version
Google Chrome 145.0.7632.116
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp01$ CHROME_VERSION=$(google-chrome --version | awk '{print $3}' | cut -d'.' -f1-3)
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp01$ sudo apt install jq
[sudo] password for vaishnavi:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libjq1 libonig5
The following NEW packages will be installed:
  jq libjq1 libonig5
0 upgraded, 3 newly installed, 0 to remove and 104 not upgraded.
Need to get 357 kB of archives.
After this operation, 1,007 kB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu jammy/main amd64 libonig5 amd64 6.9.7.1-2build1 [172 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 libjq1 amd64 1.6-2.1ubuntu3.1 [133 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 jq amd64 1.6-2.1ubuntu3.1 [52.5 kB]
Fetched 357 kB in 5s (70.0 kB/s)
Selecting previously unselected package libonig5:amd64.
(Reading database ... 247698 files and directories currently installed.)
Preparing to unpack .../libonig5_6.9.7.1-2build1_amd64.deb ...
Unpacking libonig5:amd64 (6.9.7.1-2build1) ...
Selecting previously unselected package libjq1:amd64.
Preparing to unpack .../libjq1_1.6-2.1ubuntu3.1_amd64.deb ...
Unpacking libjq1:amd64 (1.6-2.1ubuntu3.1) ...
Selecting previously unselected package jq.
Preparing to unpack .../jq_1.6-2.1ubuntu3.1_amd64.deb ...
Unpacking jq (1.6-2.1ubuntu3.1) ...
Setting up libonig5:amd64 (6.9.7.1-2build1) ...
Setting up libjq1:amd64 (1.6-2.1ubuntu3.1) ...
Setting up jq (1.6-2.1ubuntu3.1) ...
Processing triggers for man-db (2.10.2-1) ...

```

`mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false`

```

Activities Terminal Mar 4 21:38
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp01

vaishnavi@vaishnavi-VirtualBox: $ cd devops21
vaishnavi@vaishnavi-VirtualBox: ~/devops21$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp01 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> mvn archetype-plugin(3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO] <<< mvn archetype-plugin(3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- mvn-archetype-plugin(3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 kB at 1.6 MB/s)
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/vaishnavi/devops21
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenSeleniumApp01
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/vaishnavi/devops21/MyMavenSeleniumApp01
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 12.417 s
[INFO] Finished at: 2026-03-04T21:36:23+05:30
[INFO] -----
vaishnavi@vaishnavi-VirtualBox: ~/devops21$ cd MyMavenSeleniumApp01
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp01$ ls
pom.xml
vaishnavi@vaishnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp01$ google-chrome --version
Google Chrome 145.0.7632.116

```

The command can be broken down as:

DgroupId=com.example – project's base package (reverse-domain style).

-DartifactId=MyMavenSeleniumApp – defines the name of the project (will also be the folder name).

-DarchetypeArtifactId=maven-archetype-quickstart – sets the Maven template to be the basic Java application.

-DinteractiveMode=false – deactivates interactive mode and uses the given values automatically.

Step 2:

Use \$cd command to go into the directory.

```
$gedit pom.xml
```

pom.xml file is created to configure the Maven project. It defines the project information, adds required dependencies like Guava and Commons IO libraries, and includes build plugins to compile the Java code, run unit tests, and package the application into an executable JAR file.

```
<dependencies>
  <dependency>

    <groupId>junit</groupId>

    <artifactId>junit</artifactId>

    <version>3.8.1</version>

    <scope>test</scope>

  </dependency>

  <!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->

<dependency>

  <groupId>org.seleniumhq.selenium</groupId>

  <artifactId>selenium-java</artifactId>

  <version>4.29.0</version>

</dependency>
</dependencies>

<!-- Build: Configuring plugins and build settings -->
<build>
  <plugins>

    <!-- Example: Maven Compiler Plugin to compile Java code -->

    <plugin>

      <groupId>org.apache.maven.plugins</groupId>

      <artifactId>maven-compiler-plugin</artifactId>

      <version>3.8.1</version>
```

```

</configuration>

<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>

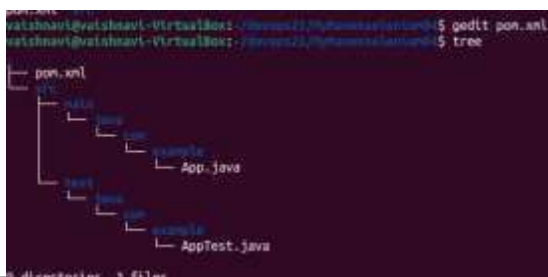
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-jar-plugin</artifactId>
    <version>3.0.1</version>
    <configuration>
        <archive>
            <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
        </archive>
    </configuration>
</plugin>

</plugins>
</build>
</project>

```

Execute \$tree command to view the structure of the project



```

vatsava@vatsava-VirtualBox: /devnull/MyMavenExample$ gedit pom.xml
vatsava@vatsava-VirtualBox: /devnull/MyMavenExample$ tree
.
├── pom.xml
├── src
│   ├── main
│   │   ├── java
│   │   │   └── com
│   │   │       └── example
│   │           └── App.java
│   └── test
│       ├── java
│       │   └── com
│       │       └── example
│       │           └── AppTest.java
└── target

```

inspect the SauceDemo webpage to find the element IDs required for automation.

Right-click on the webpage and select Inspect. The HTML code will be displayed. Click on the cursor icon (arrow symbol) available on the top-left of the inspect panel, then hover over the username field to identify its ID, which is user-name.

Similarly, check the password field (ID: password) and the login button (ID: login-button). Make a note of these IDs, as they will be used in the App.java file.

Next,

Edit the App.java file

```
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

/** * Hello world! * */

public class App
{
    public static void main( String[] args )
    {
        WebDriver driver=new ChromeDriver();
        driver.get("https://www.saucedemo.com/");
        driver.manage().window().maximize();
        driver.findElement(By.id("user-name")).sendKeys("standard_user");
        driver.findElement(By.id("password")).sendKeys("secret_sauce");
        driver.findElement(By.id("login-button")).click();
    }
}
```

Step 4:

Build and run the maven application

Execute the command \$mvn clean install

```
Activities Terminal Mar 6 21:30 valshnavi@valshnavi-VirtualBox: ~/devops21/MyMavenSeleniumApp01
```

```
Setting up jq (1.8-2ubuntu3.1) ...  
Processing triggers for man-db (2.10.2-1) ...  
  
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...#####  
jq: error: $LATEST_DRIVER_VERSION: curl -s "https://googlechromelabs.github.io/chrome-for-testing/latest-patch-versions-per-build.json" | jq -r ".builds[\"$SCROME_VERSION\"]|.version")  
Command 'curl' not found, but can be installed with:  
sudo snap install curl # version 8.18.0, or  
sudo apt install curl # version 7.81.0-ubuntu1.22  
See 'snap info curl' for additional versions.  
valshnavi@valshnavi-VirtualBox:~/devops21/MyMavenSeleniumApp01$ sudo apt install curl  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
libcurl4  
The following NEW packages will be installed:  
curl  
The following packages will be upgraded:  
libcurl4  
1 upgraded, 1 newly installed, 0 to remove and 183 not upgraded.  
Need to get 484 kB of archives:  
After this operation, 455 kB of additional disk space will be used.  
Do you want to continue? [Y/n] y  
Get:1 http://in.archive.ubuntu.com/ubuntu janny-updates/main amd64 libcurl4 amd64 7.81.0-ubuntu1.22 [296 kb]  
Get:2 http://in.archive.ubuntu.com/ubuntu janny-updates/main amd64 curl amd64 7.81.0-ubuntu1.22 [194 kb]  
Fetched 484 kB in 1s (58.7 kB/s)  
(Reading database ... 247707 files and directories currently installed.)  
Preparing to unpack .../libcurl4_7.81.0-ubuntu1.22_amd64.deb ...  
Unpacking libcurl4:amd64 (7.81.0-ubuntu1.22) over (7.81.0-ubuntu1.23) ...  
Selecting previously unselected package curl.  
Preparing to unpack .../curl_7.81.0-ubuntu1.22_amd64.deb ...  
Unpacking curl (7.81.0-ubuntu1.22) ...  
Setting up libcurl4:amd64 (7.81.0-ubuntu1.22) ...  
Setting up curl (7.81.0-ubuntu1.22) ...  
Processing triggers for man-db (2.10.2-1) ...  
Processing triggers for libc-bin (2.35-0ubuntu3.13) ...  
valshnavi@valshnavi-VirtualBox:~/devops21/MyMavenSeleniumApp01$
```

If the command shoes output as “build success”, it means that the project has compiled successfully

Step 5:

Execute `$ java -jar target/MyMavenSeleniumApp01-1.0-SNAPSHOT`

```
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenSeleniumApp01$ java -jar target/MyMavenSeleniumApp01-1.0-SNAPSHOT
Error: Unable to access jarfile target/MyMavenSeleniumApp01-1.0-SNAPSHOT
vaishnavi@vaishnavi-virtualbox:~/devops21/MyMavenSeleniumApp01$
```

If we get error in executing the above jar file do run the below command

```
$mvn exec:java -Dexec.mainClass="com.example.App"
```

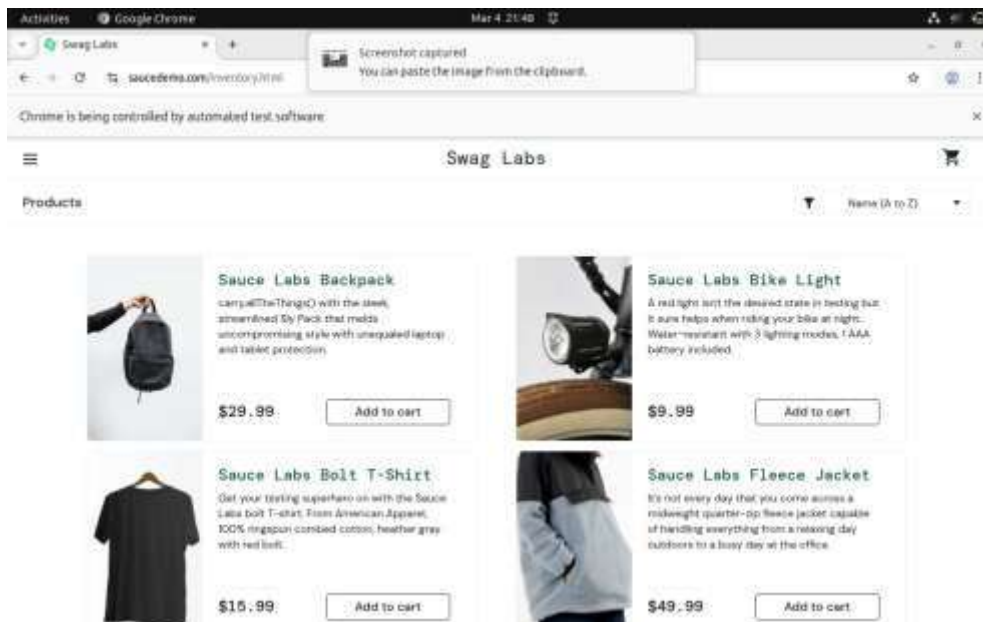
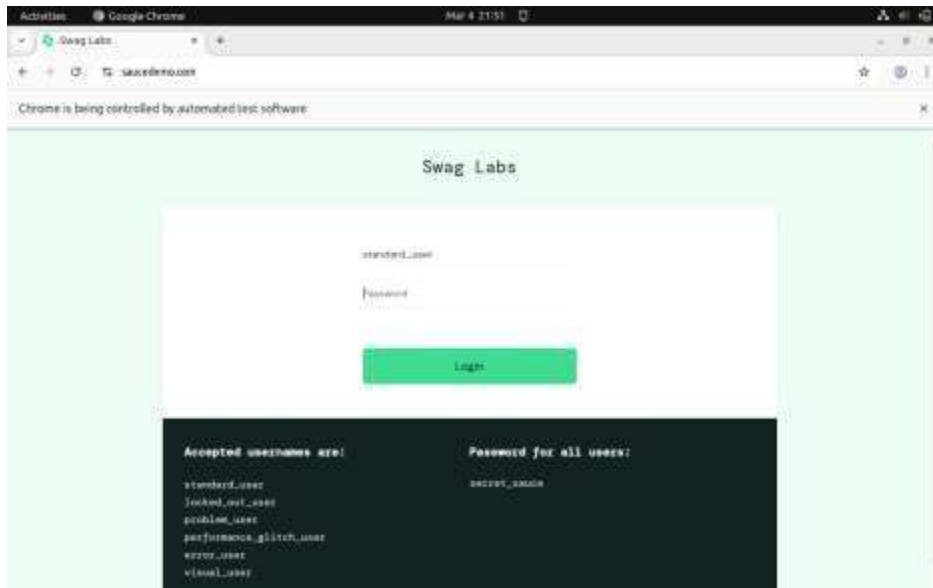
[illegible]

Sauce Demo Website

Sauce Demo (<https://www.saucedemo.com/>) is a fake e-commerce website created by Sauce Labs for testing and learning web automation tools like Selenium.

It is used for:

- Writing automation scripts and UI tests
- Demonstrating cloud testing features
- Learning and practicing test automation



Pushing The Project into Git:

Execute the commands:

```
$ git config --global user.name "vaishnavim1121"
```

```
$ git config --global user.email "vaishnavim1121@gmail.com"
```

```
$git init
```

```
$git add .
```

```
$git commit -m "initial commit"
```

```
$ git remote add origin https://github.com/vaishnavim1121/MyMavenSeleniumApp.git
```

```
$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenSeleniumApp
```

```
$ git push -u origin master
```

```

hint: See 'git help push' for more information about pushing.
hint: Pushing the master branch can be done via the command:
hint: $ git push
hint: $ git branch -m master
Initialized empty Git repository in /home/vaishnavi/devops21/MyMavenSeleniumApp3/.git/
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git add .
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git commit -m "first commit"
[master (root-commit) 1affab] first commit
13 files changed, 265 insertions(+)
create mode 100644 pom.xml
create mode 100644 src/main/java/com/example/App.java
create mode 100644 src/test/java/com/example/AppTest.java
create mode 100644 target/MyMavenSeleniumApp3-1.0-SNAPSHOT.jar
create mode 100644 target/classes/com/example/App.class
create mode 100644 target/maven-archiver/pom.properties
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
create mode 100644 target/surefire-reports/TEST-com.example.AppTest.xml
create mode 100644 target/surefire-reports/com.example.AppTest.txt
create mode 100644 target/test-classes/com/example/AppTest.class
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git config --global user.name "vaishnavim1121"
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git config --global user.email "vaishnavim1121@gmail.com"
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git remote add origin https://github.com/vaishnavim1121/MyMavenSeleniumApp3.git
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ set-url origin https://ghp_WMyMwSSxgt2QxFOV7noCjtJaL6ZV3MVFfI@github.com/vaishnavim1121/MySeleniumApp3
set-url: command not found
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git remote set-url origin https://ghp_WMyMwSSxgt2QxFOV7noCjtJaL6ZV3MVFfI@github.com/vaishnavim1121/MySeleniumApp3
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git push -u origin master
remote: Repository not found.
fatal: repository 'https://github.com/vaishnavim1121/MySeleniumApp3/' not found
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git remote set-url origin https://ghp_WMyMwSSxgt2QxFOV7noCjtJaL6ZV3MVFfI@github.com/vaishnavim1121/MyMavenSeleniumApp3
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $ git push -u origin master
Enumerating objects: 39, done.
Counting objects: 100% (39/39), done.
Delta compression using up to 10 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (39/39), 8.46 KiB | 721.00 KiB/s, done.
Total 39 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/vaishnavim1121/MyMavenSeleniumApp3
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
vaishnavi@vaishnavi-VirtualBox: /devops21/MyMavenSeleniumApp3 $

```


Second Phase

Depicting Jenkins Pipeline

Follow the given steps to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository

go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.

New Item

Enter an item name

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

OK

This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page

Now we need to add a Jenkinsfile into our project which will consist of the operations that will occur when building the project in Jenkins

It includes step-wise instructions guiding at each stage with right commands

```

pipeline {
    agent any // Use any available agent

    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/vaishnavi1121/MyMavenSeleniumApp01.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package' // Run Maven build
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test' // Run unit tests
            }
        }

        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar'
            }
        }
    }
}

```

Now try building the project with Jenkins

Stage View

	Declarative: Checkout SCM	Declarative: Tool install	Checkout	Build	Test	Run Application	Declarative: Post Actions
Average stage times: (Full run time: ~1min 2s)	3s	523ms	1s	24s	9s	12s	180ms
Apr 10 18:10	3s	523ms	1s	24s	9s	12s	180ms

Permalinks

- Last build (W1), 26 days ago
- Last stable build (W1), 26 days ago
- Last successful build (W1), 26 days ago
- Last completed build (W1), 26 days ago

REST API: Jenkins 2.562